

UD2: Programación Multihilo

Sincronización de Hilos

Sincronización de Hilos

- A menudo los hilos necesitan comunicarse unos con otros, la forma de comunicarse consiste habitualmente en **compartir** un objeto.
- Este objeto compartido lo reciben todos los hilos que lo comparten en su constructor.
- Cuando se comparte una variable, puede ocurrir que dependiendo de en qué orden se ejecuten las instrucciones de los varios hilos, el resultado obtenido sea diferente.
- Para evitarlo debemos sincronizar nuestro código.

Programación de Servicios y Procesos

Sincronización de Hilos

- **Sincronización de un método**

```
public synchronized void retirarDinero(int cantidad);
```

Cuando un hilo invoca un método ***synchronized***, trata de tomar el bloqueo del objeto al que pertenezca.

Si está libre, lo toma y se ejecuta.

Si el bloqueo está tomado por otro hilo, se suspende el segundo hasta que el primero finalice y libere el bloqueo

Programación de Servicios y Procesos

Sincronización de Hilos

Ejemplo:

Imaginemos la situación de que dos personas comparten una cuenta bancaria, y pueden sacar dinero de ella en cualquier momento.

Antes de retirar dinero se comprueba siempre si existe saldo.

La cuenta tiene 40€. Una de las personas quiere retirar 40€ y la otra 30€.

La primera llega al cajero, revisa el saldo, comprueba que hay dinero y se prepara para retirar el dinero.

Pero antes de retirarlo llega la otra persona a otro cajero, comprueba el saldo que todavía muestra los 40€ y también se dispone a retirar el dinero.

Las dos personas retiran el dinero, pero entonces el saldo actual será de -10€.

Programación de Servicios y Procesos

Sincronización de Hilos

```
public class Cuenta {
    private int saldo;
    public Cuenta(int saldo) {
        this.saldo = saldo;
    }
    public void restar(int cantidad){
        saldo-=cantidad;
    }
    public synchronized void retirarDinero(int cantidad, String nombre){
        if(saldo >= cantidad) {
            System.out.println("Se va a retirar saldo (Actual=" + saldo + ")");
            try {
                Thread.sleep(500);
            } catch (InterruptedException e) { }
            restar(cantidad);
            System.out.println(nombre + " retira " + cantidad + "(Actual=" + saldo + ")");
        } else
            System.out.println(nombre+" no puede retirar dinero (Actual=" + saldo + ")");
    }
}
```

Sincronización de Hilos

```
public class Persona extends Thread{
    private Cuenta c;
    String nombre;
    public Persona(Cuenta c, String nombre) {
        this.c = c;
        this.nombre = nombre;
    }
    public void run(){
        for(int i=0; i<4; i++){
            c.retirarDinero(10, nombre);
        }
    }
}

public class Principal {
    public static void main(String[] args) {
        Cuenta c = new Cuenta(40);
        Persona h1 = new Persona(c, "Ana");
        Persona h2 = new Persona(c, "Mikel");
        h1.start();
        h2.start();
    }
}
```